Министерство образования и науки Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет программной инженерии и компьютерной техники (ПИКТ)

Направление подготовки (специальность) — 09.03.04
(Нейротехнологии и программирование)

Операционные системы

Лабораторная работа FUZE

Выполнил студент
Исаев Сергей Алексеевич
Группа №P3323
Преподаватель: Бойко Владислав Алексеевич

г. Санкт-Петербург
2026 г.

Оглавление

1. Задание 3

2. Возможности и ограничения 3

3. Среда разработки и используемые средства 4

4. Логическая структура виртуальной файловой системы 4

5. Реализация..... 5

6. Порядок запуска и демонстрации 6

7. Тестирование 8

8. Использование LLM 9

9. Ограничения реализации 9

10. Вывод.....10

1. Задание

Изучить механизм FUSE (Filesystem in Userspace) и разработать файловую систему, работающую в пользовательском пространстве Linux. В качестве внешнего сервиса выбран GitHub. Необходимо предоставить доступ к репозиториям, веткам, коммитам и файлам через стандартные команды Linux, а также реализовать создание веток и публикацию изменений.

2. Возможности и ограничения

Разработанная система позволяет просматривать публичные репозитории указанного пользователя, их ветки и историю коммитов. Для каждого коммита доступны служебная информация, список изменённых файлов, патч и содержимое новых версий файлов. Для каждой ветки предоставляется рабочая область workspace, в которой можно создавать, изменять и удалять файлы.

Создание новой директории в branches приводит к созданию реальной ветки на GitHub. Изменения в workspace сначала сохраняются локально, а публикация выполняется отдельной командой mkdir в каталоге _commit. При изменении ветки в удалённом репозитории FUSE-система отказывается от публикации устаревшего коммита и сохраняет черновик.

Подробное описание возможностей и ограничений приведено в README.md и документах каталога docs/ репозитория.

3. Среда разработки и используемые средства

Компонент	Используемое средство / сведения
Операционная система	Ubuntu 24.04.5 LTS (гостевая ОС VirtualBox)
Язык программирования	Python 3.12.3
Работа с FUSE	pyfuse3; /dev/fuse; fusermount3
Асинхронный цикл	trio
Внешний сервис	GitHub REST API по HTTPS; urllib
Рабочий каталог	~/os-labs/fuse-github/fuse-github-v2
Точка монтирования	Каталог mnt/ в корне проекта

4. Логическая структура виртуальной файловой системы

После монтирования репозитории GitHub становятся виртуальными каталогами. Внутри каждого репозитория находятся сведения о нём и каталог branches, содержащий ветки.

```
mnt/
├── HELP.txt
└── <repository>/
    ├── info.txt
    ├── README.md
    └── branches/<branch>/
        ├── HEAD.txt
        ├── commits/<sha>/
        │   ├── INFO.txt, CHANGES.txt, diff.patch
        │   └── changed/
        ├── commits/pages/<number>/
        ├── workspace/
        ├── _commit/<message>/SHA.txt
        └── _merge/
            └── <source-branch>/
                └── SHA.txt
```

Каталог `commits` содержит историю коммитов, `changed` — только файлы, изменённые соответствующим коммитом, `workspace` — текущее состояние ветки с локальными изменениями. Каталог `_commit` служит для публикации нового коммита. Назначение всех объектов подробнее описано в `docs/VIRTUAL_FS.md`.

5. Реализация

5.1. Основные модули

Файл	Назначение
fuse_github.py	Формирование виртуального дерева, обработчики операций FUSE, работа с ветками и коммитами.
github_api.py	HTTP-запросы к GitHub REST API, авторизация, чтение и создание объектов Git.
workspace_store.py	Сохранение локальных изменений и сведений об опубликованных коммитах вне точки монтирования.

Класс `GitHubFS` наследует pyfuse3.Operations. Он хранит узлы виртуальной файловой системы с inode и обрабатывает обращения ядра Linux. Чтение и запись файлов выполняются через соответствующие обработчики, а сетевые запросы передаются модулю `github_api.py`.

5.2. Основные операции FUSE

Операция	Роль в файловой системе
getattr, lookup	Получение атрибутов виртуальных объектов и поиск по имени.
opendir, readdir	Открытие каталогов и выдача списка записей.
open, read	Открытие и чтение файлов, в том числе исторических версий.
create, write, unlink	Создание, изменение и удаление файлов в workspace.
mkdir	Создание ветки, каталога workspace или публикация коммита в зависимости от пути.

Данные подгружаются по мере обращения к каталогам и файлам. Например, история ветки запрашивается при открытии `commits`, а содержимое Git blob - при чтении соответствующего файла. Это уменьшает количество запросов к GitHub.

5.3. Создание коммита

При работе с `workspace` файловые операции сначала изменяют локальный черновик. Для его публикации пользователь создаёт каталог с именем сообщения коммита внутри `_commit`. Система проверяет актуальный HEAD ветки, создаёт необходимые Git blobs и дерево, формирует новый коммит и обновляет ссылку ветки без force push.

Если удалённый HEAD изменился, возвращается ошибка ESTALE (Stale file handle). Неопубликованные файлы при этом сохраняются локально. Черновик также сохраняется после остановки и повторного запуска FUSE.

5.4. Слияние и удаление веток

Для слияния веток используется специальный каталог `_merge`. При создании в нём директории с именем исходной ветки система отправляет запрос к GitHub REST API для объединения изменений с целевой веткой. Перед выполнением операции проверяется состояние рабочей области и актуальность HEAD. Если на GitHub возникает конфликт слияния, операция отклоняется без автоматического разрешения конфликта.

Удаление ветки выполняется командой rmdir в каталоге `branches`. При этом запрещено удаление ветки по умолчанию и ветки с неопубликованными локальными изменениями. После успешного запроса к GitHub удалённая ветка исключается из виртуального дерева файловой системы.

6. Порядок запуска и демонстрация работы файловой системы

Для проверки реализованной файловой системы использовалась операционная система Ubuntu и публичный тестовый репозиторий tisak142/fuse-lab-test на GitHub. Основные операции выполнялись через стандартные команды Linux, без использования команд Git CLI.

6.1. Запуск файловой системы

Перед запуском было активировано виртуальное окружение Python. Затем выполнено монтирование файловой системы с указанием пользователя GitHub и максимального количества отображаемых репозиториев.

```
cd ~/os-labs/fuse-github/fuse-github-v2
source .venv/bin/activate
python3 fuse_github.py --username tisak142 --limit 30
```

После успешного запуска программа создала точку монтирования mnt и предоставила доступ к публичным репозиториям выбранного пользователя.

В другом терминале были выполнены команды:

```
ls mnt
ls mnt/fuse-lab-test
```

Команда ls mnt отображает список доступных репозиториев, каждый из которых представлен отдельным виртуальным каталогом. При переходе в каталог конкретного репозитория можно получить информацию о нём, прочитать README и перейти к списку веток.

```
(.venv) isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ python3 fuse_github.py --username tisak142 --limit 30
Mounted @tisak142 (10 repos) at /home/isaev142/os-labs/fuse-github/fuse-github-v2/mnt
Drafts outside mount: /home/isaev142/os-labs/fuse-github/fuse-github-v2/.fuse_github_state
Unmount with: fusermount3 -u '/home/isaev142/os-labs/fuse-github/fuse-github-v2/mnt'
```

Рисунок 1 - Монтирование файловой системы

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ ls -la mnt
total 1
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 csa_lab3_scheme
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 fuse-lab-test
-r--r--r-- 1 isaev142 isaev142 713 Nov  7 20:46 HELP.txt
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 ITMO_Programming-Lab5
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 ITMO_Programming-Lab6
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 Lab4
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 OPI
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 Programming_ITMO_Lab3-4
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 ReservationSystem-studing-
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 RGR_MatStat
dr-xr-xr-x 3 isaev142 isaev142   0 Nov  7 20:46 Weblab3
```

Рисунок 2 - отображение репозиториев GitHub

.

6.2. Просмотр веток и истории коммитов

Для просмотра веток тестового репозитория была выполнена команда:

```
ls mnt/fuse-lab-test/branches
```

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ ls -la mnt/fuse-lab-test/branches
total 0
dr-xr-xr-x 5 isaev142 isaev142 0 Nov  7 20:47 feature-demo
dr-xr-xr-x 2 isaev142 isaev142 0 Nov  7 20:47 main
```

Рисунок 3 - вывод веток репозитория.

В результате отображаются каталоги, соответствующие веткам GitHub. Для каждой ветки доступны сведения о текущем HEAD, история коммитов и рабочая область.

Для получения истории основной ветки были выполнены команды:

```
R=mnt/fuse-lab-test/branches
ls "$R/main/commits"
```

Каталог `commits` содержит идентификаторы коммитов SHA, представленные в виде отдельных виртуальных директорий.

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ R=mnt/fuse-lab-test/branches
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ ls -la "$R/main/commits"
total 0
dr-xr-xr-x 2 isaev142 isaev142 0 Nov  8 12:45 1f0a5d3757b433440b1dee56b1b0ae2c938c7ad4
dr-xr-xr-x 3 isaev142 isaev142 0 Nov  8 12:45 pages
```

Рисунок 4 - вывод коммитов конкретной ветки.

Для получения информации о последнем коммите использовались команды:

```
SHA=$(cat "$R/main/HEAD.txt")
cat "$R/main/commits/$SHA/INFO.txt"
cat "$R/main/commits/$SHA/CHANGES.txt"
```

Файл `INFO.txt` содержит SHA, сведения об авторе, дату, сообщение коммита и его родительские коммиты. Файл `CHANGES.txt` предоставляет список добавленных, изменённых и удалённых файлов. Дополнительно через `diff.patch` можно посмотреть различия между версиями файлов, а через `changed/` — содержимое файлов после выбранного коммита.

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ cat "$R/main/commits/$SHA/INFO.txt"
SHA: 1f0a5d3757b433440b1dee56b1b0ae2c938c7ad4
Author: Isaev Sergei <isaev1412@yandex.ru>
Date: 2026-11-07T17:02:37Z
Message: Initial commit
Parents: (root commit)
Comparison: first parent for merges; GitHub API commit file list.
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ cat "$R/main/commits/$SHA/CHANGES.txt"
ADDED README.md
```

Рисунок 5 - вывод коммитов конкретной ветки.

6.3. Создание новых веток

Для проверки создания веток использовалась команда `mkdir` в виртуальном каталоге `branches`. Были созданы отдельные ветки для редактирования файлов, последующего слияния и удаления.

```
mkdir "$R/report-source"
```

```
mkdir "$R/report-target"
mkdir "$R/report-delete"
ls "$R"
```

В результате выполнения команд в виртуальном каталоге появились новые ветки. При этом система отправила соответствующие запросы к GitHub REST API, поэтому созданные ветки также стали доступны на сайте GitHub.

Таким образом, операция `mkdir` используется не только для создания обычных каталогов, но и для управления ветками удалённого репозитория.

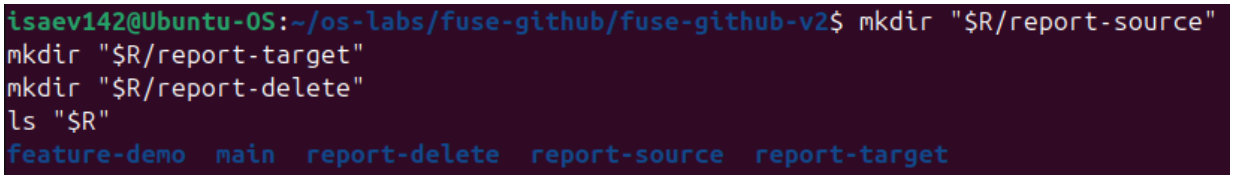


Рисунок 6 - Создание веток через виртуальную файловую систему.

6.4. Работа с файлами в workspace

Для изменения содержимого ветки используется каталог workspace, представляющий её рабочее состояние. В отличие от исторических коммитов, файлы в этом каталоге доступны для создания, изменения и удаления.

Для демонстрации была выбрана ветка `report-source`. В её рабочей области созданы два текстовых файла.

```
S="$R/report-source"

printf 'Version 1\n' > "$S/workspace/demo.txt"
printf 'Temporary file\n' > "$S/workspace/remove-me.txt"

ls -l "$S/workspace"
cat "$S/workspace/demo.txt"
```

После выполнения команд новые файлы появились в виртуальной рабочей области. Их содержимое стало доступно для чтения через `cat`.

При этом изменения первоначально сохраняются в локальном черновике файловой системы и не публикуются на GitHub автоматически. Это позволяет подготовить несколько изменений и объединить их в один коммит.

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ S="$R/report-source"

printf 'Version 1\n' > "$S/workspace/demo.txt"
printf 'Temporary file\n' > "$S/workspace/remove-me.txt"

ls -l "$S/workspace"
cat "$S/workspace/demo.txt"

total 2
-rw-r--r-- 1 isaev142 isaev142 10 Nov  8 12:51 demo.txt
-rw-r--r-- 1 isaev142 isaev142 15 Nov  8 12:51 README.md
-rw-r--r-- 1 isaev142 isaev142 15 Nov  8 12:51 remove-me.txt
Version 1
```

Рисунок 7 - Создание и чтение файлов в рабочей области ветки.

6.5. Создание коммитов и изменение файлов

Для публикации подготовленных файлов была выполнена команда:

```
mkdir "$S/_commit/01-add-demo"
cat "$S/_commit/01-add-demo/SHA.txt"
```

Создание каталога внутри `_commit` инициирует публикацию изменений. Система формирует необходимые объекты Git и обновляет HEAD соответствующей ветки. После успешного выполнения созданному коммиту соответствует уникальный SHA, доступный в файле `SHA.txt`.

Для проверки остальных файловых операций содержимое `demo.txt` было изменено, файл `remove-me.txt` удалён, а файл `new.txt` создан.

```
printf 'Version 2\n' > "$S/workspace/demo.txt"
rm "$S/workspace/remove-me.txt"
printf 'New file\n' > "$S/workspace/new.txt"
```

После подготовки изменений создан второй коммит:

```
mkdir "$S/_commit/02-edit-demo"
SHA=$(cat "$S/_commit/02-edit-demo/SHA.txt")

cat "$S/commits/$SHA/CHANGES.txt"
cat "$S/commits/$SHA/changed/demo.txt"
```

`B CHANGES.txt` должны отображаться операции MODIFIED, REMOVED и ADDED, соответствующие изменению, удалению и добавлению файлов.

Каталог `changed` позволяет получить новую версию изменённого файла непосредственно из истории GitHub.

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ printf 'Version 2\n' > "$S/workspace/demo.txt"
rm "$S/workspace/remove-me.txt"
printf 'New file\n' > "$S/workspace/new.txt"
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ mkdir "$S/_commit/02-edit-demo"
SHA=$(cat "$S/_commit/02-edit-demo/SHA.txt")

cat "$S/commits/$SHA/CHANGES.txt"
cat "$S/commits/$SHA/changed/demo.txt"
ADDED demo.txt
ADDED new.txt
Version 2
```

Рисунок 8 - Публикация коммита и просмотр изменений файлов.

6.6. Слияние веток

Для объединения изменений двух веток используется специальный виртуальный каталог `_merge`. В нём создание директории с именем исходной ветки инициирует операцию слияния через GitHub REST API.

В рамках демонстрации изменения из ветки `report-source` объединяются с веткой `report-target`.

```
T="$R/report-target"

mkdir "$T/_merge/report-source"
cat "$T/_merge/report-source/SHA.txt"
cat "$T/HEAD.txt"
```

В данной операции `report-source` является исходной веткой, а `report-target` — целевой. После успешного слияния HEAD целевой ветки обновляется, а файлы, опубликованные в исходной ветке, становятся доступны в целевой.

GitHub выполняет слияние на стороне сервера. Если изменения веток конфликтуют, операция отклоняется без автоматического разрешения конфликта.

```
isaev142@Ubuntu-OS:~/os-labs/fuse-github/fuse-github-v2$ T="$R/report-target"

mkdir "$T/_merge/report-source"
cat "$T/_merge/report-source/SHA.txt"
cat "$T/HEAD.txt"
f4d13931dffece4aaad0537207dd91316862f6f5
f4d13931dffece4aaad0537207dd91316862f6f5
```

Рисунок 6 — Слияние веток через FUSE и GitHub REST API.

6.7. Удаление ветки

Удаление ветки реализовано через стандартную команду `rmdir`. Для демонстрации использовалась ранее созданная отдельная ветка `report-delete`, не содержащая неопубликованных локальных изменений.

Сначала был получен список существующих веток, затем выполнено удаление.

```
ls "$R"
rmdir "$R/report-delete"
ls "$R"
```

После успешного выполнения операции ветка `report-delete` исчезает из виртуальной файловой системы и удаляется с GitHub.

В реализации предусмотрены ограничения безопасности: нельзя удалить ветку по умолчанию или ветку с неопубликованными локальными изменениями.

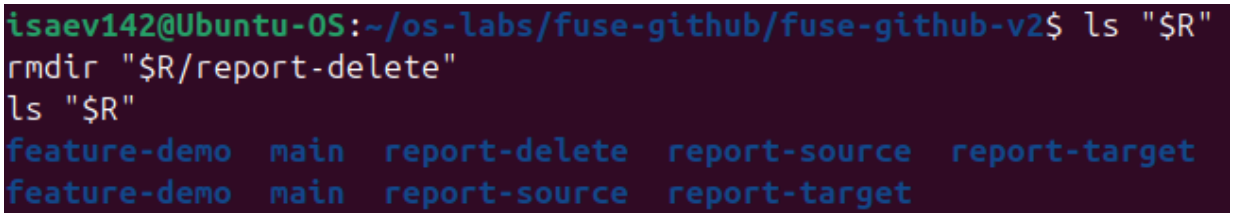


Рисунок 7 — Удаление ветки GitHub командой rmdir.

6.8. Сохранение черновиков и защита от конфликтов

В файловой системе предусмотрено локальное сохранение неопубликованных изменений. Для проверки был создан файл `draft.txt`, после чего файловая система была размонтирована и запущена повторно.

После повторного монтирования команда чтения файла вернула его прежнее содержимое:

```
cat mnt/fuse-lab-test/branches/feature-demo/workspace/draft.txt
```

Результат:

```
Draft survives restart
```

Это подтверждает, что локальный черновик сохраняется вне точки монтирования и может быть восстановлен после перезапуска программы.

Также была проверена защита от конфликтов при параллельном изменении ветки. После изменения HEAD непосредственно через GitHub была выполнена попытка опубликовать ранее подготовленный черновик.

```
mkdir mnt/fuse-lab-test/branches/feature-demo/_commit/conflict-test
```

Файловая система отклонила операцию с ошибкой:

```
Stale file handle
```

При этом конфликтующий коммит не был опубликован, а локальный черновик сохранился. Таким образом, система предотвращает публикацию изменений поверх неожиданно обновлённой удалённой ветки.

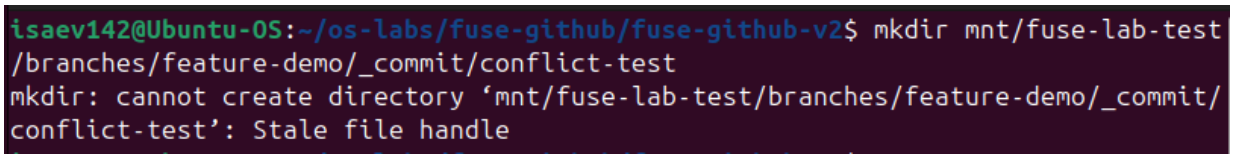


Рисунок 8 - Обработка конфликта HEAD.

6.9. Размонтирование файловой системы

После завершения работы файловая система отключается с помощью утилиты `fusermount3`.

```
cd ~/os-labs/fuse-github/fuse-github-v2
fusermount3 -u ./mnt
```

Для проверки можно использовать команду:

```
mountpoint ./mnt
```

После успешного размонтирования каталог `mnt` перестаёт быть точкой монтирования FUSE. При этом локальные черновики, сохранённые вне `mnt`, не удаляются.

В ходе демонстрации были рассмотрены основные операции файловой системы: просмотр репозиториев, веток и истории коммитов, создание и изменение файлов, публикация коммитов, слияние и удаление веток, а также сохранение локальных изменений и обработка конфликтов.

7. Тестирование

Проверка проводилась на публичных репозиториях OPI и Hello-World, а операции записи - в отдельном тестовом репозитории fuse-lab-test. Основные результаты приведены в таблице.

Проверяемая возможность	Результат
Просмотр репозиториев, веток и истории	Каталоги и SHA коммитов доступны через ls.
Добавленные и изменённые файлы	В CHANGES.txt отображаются ADDED и MODIFIED.

Создание, изменение и удаление файла	Все операции опубликованы отдельными коммитами.
Удаление в истории	Получена запись REMOVED hello.txt.
Сохранение черновика	Файл draft.txt остался после перезапуска FUSE.
Защита от конфликта HEAD	Получена ошибка ESTALE; конфликтующий коммит не опубликован.
Постраничная навигация	commits/ и pages/1 совпадают; при пяти коммитах pages/2 отсутствует.
Метаданные исторического файла	stat и wc -c показали 35 байт для README.md.

8. Использование LLM при разработке

В ходе выполнения лабораторной работы использовалась большая языковая модель ChatGPT для подготовки модуля взаимодействия с GitHub REST API. Код обращений к внешнему сервису выделен в самостоятельный файл `github_api.py`, отдельно от обработчиков FUSE.

Полученный модуль проверялся через тесты с имитацией ответов API и при реальной работе файловой системы. При тестировании в Ubuntu были проверены не только запросы на чтение, но и операции записи в собственный GitHub-репозиторий.

9. Ограничения реализации

Для одного запуска выбирается от 1 до 30 публичных репозиториев. В истории первая страница содержит до 30 коммитов, последующие доступны через pages. Отображается не более 500 веток и не более 4000 элементов дерева workspace; размер обычного файла ограничен 2 МиБ.

Подробное руководство по запуску доступно в `README.md` и `docs/USAGE.md`. Исходный код, тесты и документация в GitHub-репозитории проекта.

10. Вывод

В ходе выполнения лабораторной работы я изучил принцип работы FUSE и механизм взаимодействия файловой системы в пользовательском пространстве с ядром Linux. На практике познакомился с библиотекой pyfuse3, асинхронной обработкой файловых операций и организацией виртуальных inode.

Я реализовал файловую систему, через которую можно просматривать публичные репозитории GitHub, их ветки, коммиты и содержимое изменённых файлов с помощью обычных команд Linux. Дополнительно была реализована рабочая область для создания, изменения и удаления файлов, а также механизм публикации новых веток и Git-коммитов через REST API.

При тестировании я проверил операции чтения и записи, сохранение неопубликованных изменений после перезапуска и отказ от публикации при конфликте удалённой ветки. Работа показала, как внешнее API можно представить в виде привычной файловой структуры Linux, не используя для основных операций команды Git CLI.

Полный исходный код, инструкция пользователя, описание виртуальной структуры и результаты проверок собраны в репозитории проекта. В отчёте приведены основные этапы реализации и демонстрации работы системы.

Ссылка на репозиторий: https://github.com/tisak142/ITMO_OS_FUSE